

URL Malicious Detection

Using Ensemble Methods

A Comparative Study of Machine Learning Approaches

Khong Tuan Anh 202416775

June 18, 2026

Contents

1	Second Ensemble Method	3
1.1	Overview	3
1.2	Constants	3
1.2.1	Suspicious TLDs (SUSPICIOUS_TLDS)	4
1.2.2	Sensitive Keywords (SENSITIVE_KEYWORDS)	4
1.2.3	Target Brand Names (BRAND_NAMES)	4
1.3	URL Parser and Tokenizer	5
1.3.1	URL Structure Decomposition: parse_url(url)	5
1.3.2	Domain Analysis: tokenize_domain(url)	6
1.3.3	Path Analysis: tokenize_path(url)	8
1.4	Custom Transformers for FeatureUnion	10
1.5	Model Storage and Prediction Engine	11
1.5.1	Main Execution Workflow	12
1.5.2	Dataset Selection and Loading	12
1.5.3	Data Cleaning and Preprocessing	12
1.5.4	Dataset Splitting	13
1.6	Feature Extraction using FeatureUnion and TF-IDF	13
1.7	Logistic Regression Classifier	15
1.7.1	Foundation: How the Algorithm Interprets URLs	15
1.7.2	Step 1: Computing the Linear Risk Score	15
1.7.3	Step 2: Converting the Score into a Probability	16
1.7.4	Step 3: Model Training Using Log-Loss and L-BFGS	16
1.7.5	Parameter Configuration	17
1.8	LightGBM Classifier	18
1.8.1	Foundation: Gradient Boosting	18
1.8.2	Key Innovations of LightGBM	19
1.8.3	Parameter Configuration	19
1.8.4	Role in the Ensemble Framework	20
1.9	Ensemble Classification using Soft Voting	21
1.9.1	Soft Voting Mechanism	21
1.9.2	Weighted Decision Strategy	22
1.9.3	Rationale for Combining Logistic Regression and LightGBM	22
1.9.4	Benefits of the Ensemble Approach	22
1.10	Pipeline Integration and Model Evaluation	23
1.10.1	Pipeline Construction	23
1.10.2	Cross-Validation	23
1.10.3	Model Training and Persistence	24

1.10.4 Performance Evaluation	24
1.10.5 False Positive Analysis	25
1.11 Deployment and Model Loading	25
1.12 Model Deployment and Testing	26
1.12.1 Environment Setup	26
1.12.2 Handling Pickle Deserialization Issues	26
1.12.3 Model Loading and Validation	27
1.12.4 URL Classification Process	27
1.13 Experimental Results and Performance Analysis	28
1.13.1 Overall Model Performance	28
1.13.2 Class-Level Performance Analysis	28
1.13.3 Discussion	30
1.13.4 Future Improvements	30

1. Second Ensemble Method

1.1 Overview

For the second machine learning approach, a different strategy was adopted to train a model capable of detecting whether a given URL is safe or malicious. Unlike the first method presented previously, this approach introduces a new perspective and methodology, allowing for a comparative evaluation between different machine learning techniques.

In this ensemble learning approach, three algorithms are combined to leverage their individual strengths and improve overall classification performance:

- **Logistic Regression:** A well-established algorithm for classification tasks, particularly suitable for binary classification problems such as distinguishing between benign and malicious URLs. It provides a simple yet effective baseline model and offers good interpretability.
- **Light Gradient Boosting Machine (LightGBM):** A powerful tree-based machine learning algorithm that employs the Gradient Boosting framework. LightGBM is capable of capturing complex non-linear relationships among URL features and is widely recognized for its high predictive performance and computational efficiency.
- **Soft Voting Classifier:** This algorithm acts as an ensemble decision mechanism by combining the probability outputs of the two base models. Instead of relying on a single classifier, the Soft Voting Classifier aggregates the predictions from Logistic Regression and LightGBM, producing a final decision based on the averaged class probabilities. This approach often leads to improved robustness and classification accuracy compared to individual models.

1.2 Constants

The `CONSTANTS` section is defined at the beginning of the source code and serves as a critical component of the system. It functions as a knowledge base containing predefined rules and heuristics that allow the system to perform preliminary security checks before passing the URL to more sophisticated machine learning models.

This section consists of three core lists:

```
1 SUSPICIOUS_TLDS = {"ru", "cn", "info", "biz", "tk", "ml", "ga",  
    "cf", "gq"}
```

```
2
3 SENSITIVE_KEYWORDS = [
4     "login", "signin", "secure", "update", "verify",
5     "account", "banking", "confirm", "password", "credential",
6     "support", "helpdesk", "recover", "unlock", "alert",
7     "suspended", "unusual", "activity"
8 ]
9
10 BRAND_NAMES = [
11     "paypal", "apple", "amazon", "google", "microsoft",
12     "facebook", "netflix", "ebay", "bank", "chase"
13 ]
```

Listing 1: Predefined constant lists used for rule-based URL screening.

1.2.1 Suspicious TLDs (SUSPICIOUS_TLDS)

This list contains top-level domains (TLDs) that are frequently associated with malicious activities, including .ru, .cn, .tk, .ml, and others.

Purpose: Cybercriminals often prefer free or loosely regulated domain extensions to create phishing and malicious websites at minimal cost. Therefore, the system generates an initial warning signal whenever a URL contains one of these suspicious TLDs.

1.2.2 Sensitive Keywords (SENSITIVE_KEYWORDS)

This list includes terms such as login, secure, update, verify, password, alert, and similar security-related keywords.

Purpose: Attackers commonly use these terms to create a sense of urgency or to deceive users into believing they are interacting with a legitimate authentication or security-related webpage.

A notable design decision in the implementation is that these keywords are analyzed *exclusively within the domain name* rather than the URL path. This significantly reduces false positives for legitimate websites. For example, a domain such as secure-update.com would be flagged as suspicious, whereas a legitimate URL such as apple.com/secure-update would not trigger the same warning.

1.2.3 Target Brand Names (BRAND_NAMES)

This list contains the names of well-known organizations and online services, including PayPal, Apple, Amazon, Google, Facebook, and others.

Purpose: The list is used to support the *Brand Impersonation Detection* mechanism. The algorithm examines whether a brand name appears within a URL while the actual root domain does not belong to the corresponding organization.

For instance, if the keyword `paypal` appears in a URL such as `paypal.security-check.com`, but the root domain is not owned by PayPal, the system identifies the URL as a potential phishing or impersonation attempt and marks it as highly suspicious.

Overall, the **CONSTANTS** section provides the machine learning framework with a strong rule-based foundation derived from commonly observed cybercriminal behaviors. By incorporating domain knowledge and security heuristics into the feature extraction process, the system can identify suspicious patterns more effectively and improve the overall accuracy of malicious URL detection.

1.3 URL Parser and Tokenizer

The **URL Parser and Tokenizer** component serves as the core of the data pre-processing stage within the proposed system. Rather than directly feeding raw URL strings into machine learning models—which generally perform poorly when processing unstructured text—the system first decomposes each URL into meaningful components, extracts structural characteristics, and identifies suspicious patterns commonly associated with malicious websites.

The following sections describe the functionality and design rationale of each pre-processing function.

1.3.1 URL Structure Decomposition: `parse_url(url)`

Purpose The `parse_url()` function acts as a URL parsing module that breaks a raw URL into several independent components, allowing the system to analyze each element separately and more effectively.

Processing Workflow Data Normalization. The function first standardizes the input URL by:

- Converting all characters to lowercase.
- Removing leading and trailing whitespace.
- Stripping protocol prefixes such as `http://` and `https://`.
- Removing the `www.` prefix when present.

Domain and Path Extraction. The URL is then split at the first forward slash (/):

- The portion before the slash is identified as the **domain**.
- The remaining portion is treated as the **path**.

Domain Cleaning. To obtain a normalized domain name, the function removes query parameters beginning with ?, fragment identifiers beginning with #, and port numbers beginning with :.

Root Domain and TLD Identification. The cleaned domain is further divided using the period (.) delimiter:

- **TLD (Top-Level Domain):** The last element (e.g., `com`, `vn`, `ru`).
- **Root Domain:** The second-to-last element, representing the primary owner of the domain.

This extraction step is particularly important because it enables the system to determine the actual identity of the website. For example, given the URL:

`https://www.secure-login.paypal.com/update-account?user=123`

The extracted components are: Domain: `secure-login.paypal.com`, Path: `update-account?user=`
Root Domain: `paypal`, TLD: `com`.

```

1 def parse_url(url: str) -> dict:
2     s = str(url).strip().lower()
3     s = re.sub(r'https?://', '', s)
4     s = re.sub(r'^www\.', '', s)
5     parts = s.split('/', 1)
6     domain = parts[0].split('?')[0].split('#')[0].split(':')[0]
7     path = parts[1] if len(parts) > 1 else ''
8     dparts = domain.split('.')
9     root = dparts[-2] if len(dparts) >= 2 else dparts[0]
10    tld = dparts[-1] if dparts else ''
11    return {"domain": domain, "path": path, "root": root, "tld":
        tld}

```

Listing 2: `parse_url()` – URL structure decomposition.

1.3.2 Domain Analysis: `tokenize_domain(url)`

Purpose The `tokenize_domain()` function performs a detailed examination of the domain name. This component contains several rule-based detection mechanisms specifically designed to identify common obfuscation and social-engineering techniques employed by cybercriminals.

Feature Extraction and Token Generation URL Length Categorization.

URLs are grouped into predefined categories such as `__len_short__`, `__len_medium__`, and `__len_long__`. Excessively long URLs are often associated with phishing attacks because they can conceal the true destination domain beyond the visible area of a browser or mobile screen.

Special Character Analysis. The system evaluates the occurrence of suspicious structural patterns:

- **Multiple Hyphens** (`__many_dashes__`): Attackers commonly use hyphens to concatenate words and mimic legitimate services, e.g., `paypal-security-update.com`.
- **Multiple Dots** (`__many_dots__`): A large number of periods may indicate excessive use of subdomains, e.g., `login.apple.id.verification.com`.
- **Direct IP Address Usage** (`__ip_address__`): URLs using raw IP addresses are generally suspicious as legitimate public services rarely rely on direct IP-based access, e.g., `http://192.168.1.1`.

Sensitive Keyword Detection. The function scans the domain for predefined security-related keywords. When detected, corresponding tokens such as `__kw_login__` are generated and supplied to the machine learning pipeline.

Brand Impersonation Detection. One of the most important detection mechanisms is brand impersonation analysis. The core logic can be summarized as:

```
if brand in domain and root != brand
```

This rule verifies whether a well-known brand name appears within the domain while ensuring that the root domain actually belongs to that brand. For instance, `apple.com` is legitimate (root domain matches brand), whereas `apple-update-support.com` triggers the `__brand_impersonation__` token.

```

1 def tokenize_domain(url: str) -> list:
2     p      = parse_url(url)
3     domain = p["domain"]
4     root   = p["root"]
5     tld    = p["tld"]
6     tokens = []
7
8     full_len = len(str(url).strip())
9     if full_len < 30: tokens.append("__len_short__")
10    elif full_len < 75: tokens.append("__len_medium__")
11    elif full_len < 150: tokens.append("__len_long__")
12    else:               tokens.append("__len_verylong__")

```



```

13
14     if domain.count('-',') >= 3:                                tokens.append("__many_dashes__")
15     if domain.count('.',') >= 3:                                tokens.append("__many_dots__")
16     if sum(c.isdigit() for c in domain) >= 4:                  tokens.append("__many_digits_domain__")
17     if "@" in str(url):                                         tokens.append("__has_at__")
18     if re.match(r'^\d{1,3}(\.\d{1,3}){3}', domain): tokens.
19         append("__ip_address__")
20
21     tokens.append(f"__tld_{tld}__")
22     if tld in SUSPICIOUS_TLDS:
23         tokens.append("__suspicious_tld__")
24
25     for kw in SENSITIVE_KEYWORDS:
26         if kw in domain:
27             tokens.append(f"__kw_{kw}__")
28
29     for brand in BRAND_NAMES:
30         if brand in domain and root != brand:
31             tokens.append(f"__fake_{brand}__")
32             tokens.append("__brand_impersonation__")
33             break
34
35     dwords = re.split(r'^a-zA-Z0-9]', domain)
36     tokens.extend([w.lower() for w in dwords if len(w) > 1])
37
38     return tokens

```

Listing 3: tokenize_domain() – Domain-level feature extraction and token generation.

1.3.3 Path Analysis: tokenize_path(url)

Purpose The tokenize_path() function focuses on extracting potentially suspicious characteristics from the URL path, which is the section appearing after the domain name.

Processing Workflow Path Depth Analysis. The number of forward slashes (/) is counted to estimate the depth of the path structure. Generated tokens include

`__path_depth_shallow__` and `__path_depth_deep__`. Malicious websites occasionally conceal payloads within deeply nested directory structures.

Query Parameter Analysis. The function evaluates the presence and complexity of URL parameters:

- `__has_query__` – Indicates the presence of query parameters.
- `__many_params__` – Indicates an unusually large number of parameter assignments.

Deliberate Restriction to Reduce False Positives. A notable design decision is that the path analysis module does **not** inspect sensitive keywords or brand names. This restriction was intentionally implemented to minimize false-positive detections. For example, a legitimate technology news article may contain a path such as `tech.com/news/paypal`. Although the path contains words like *PayPal*, *login*, and *update*, the page itself is entirely legitimate. By restricting keyword and brand analysis exclusively to the domain component, the system significantly improves precision.

```

1 def tokenize_path(url: str) -> list:
2     path = parse_url(url)["path"]
3     tokens = []
4
5     if not path:
6         tokens.append("__no_path__")
7         return tokens
8
9     depth = path.count('/')
10    if depth == 0: tokens.append("__path_depth_0__")
11    elif depth <= 2: tokens.append("__path_depth_shallow__")
12    elif depth <= 5: tokens.append("__path_depth_medium__")
13    else: tokens.append("__path_depth_deep__")
14
15    if '?' in path: tokens.append("__has_query__")
16    if path.count('=') >= 3: tokens.append("__many_params__")
17
18    pwords = re.split(r'^a-zA-Z0-9', path)
19    tokens.extend([w.lower() for w in pwords if len(w) > 1])
20
21    return tokens

```

Listing 4: `tokenize_path()` – Path-level feature extraction.

Summary

The URL Parser and Tokenizer module transforms raw URLs into structured, machine-readable representations by decomposing URLs into domains and paths, extracting security-relevant indicators, and generating both rule-based and textual features. This preprocessing stage provides the foundation for subsequent TF-IDF vectorization and machine learning classification, enabling the detection models to learn meaningful patterns associated with phishing and malicious URLs while maintaining a low false-positive rate.

1.4 Custom Transformers for FeatureUnion

The project implements two custom transformers, `DomainExtractor` and `PathExtractor`, to integrate URL preprocessing into the Scikit-learn pipeline.

```
1 class DomainExtractor(BaseEstimator, TransformerMixin):
2     def fit(self, X, y=None): return self
3     def transform(self, X):
4         return [" ".join(tokenize_domain(url)) for url in X]
5
6 class PathExtractor(BaseEstimator, TransformerMixin):
7     def fit(self, X, y=None): return self
8     def transform(self, X):
9         return [" ".join(tokenize_path(url)) for url in X]
```

Listing 5: Custom Scikit-learn transformers for domain and path tokenization.

Purpose of the Transformers

In Scikit-learn, a transformer is a preprocessing component that converts raw input data into a format suitable for machine learning models. These custom transformers serve as adapters between the URL tokenization functions and the machine learning pipeline.

By inheriting from `BaseEstimator` and `TransformerMixin`, both classes become fully compatible with Scikit-learn's `Pipeline` and `FeatureUnion` frameworks, enabling seamless integration into the training workflow.

Fit Method

The `fit()` method simply returns `self` because these transformers do not learn any parameters from the training data. Their behavior is entirely rule-based and relies on predefined tokenization logic.

Transform Method

The `transform()` method performs the actual preprocessing. For each URL, it calls either `tokenize_domain()` or `tokenize_path()` to generate a list of tokens describing the URL's characteristics.

Since the subsequent `TfidfVectorizer` requires text input rather than Python lists, the generated tokens are concatenated into a single string using whitespace as a separator. For example, the token list:

```
['_len_short_', '_kw_login_', 'evil', '_tld_com_']
```

is converted into:

```
_len_short_ _kw_login_ evil _tld_com_
```

Summary

The `DomainExtractor` and `PathExtractor` classes bridge the gap between rule-based URL analysis and machine learning feature extraction. They transform tokenized URL components into a textual representation that can be processed by TF-IDF vectorization, while maintaining full compatibility with the Scikit-learn pipeline architecture.

1.5 Model Storage and Prediction Engine

The system stores the trained machine learning model using the Pickle (`.pkl`) format. This file contains the complete trained pipeline, including feature extraction components, TF-IDF vectorizers, and classification models.

The `load_model()` function loads the serialized model from disk using the `joblib` library, allowing other modules to perform predictions without retraining the model.

The `predict_urls()` function serves as the inference engine. Given a list of URLs, it performs two tasks:

- `predict()` generates the final classification result (benign or phishing).
- `predict_proba()` returns the prediction confidence score based on class probabilities.

```
1 MODEL_PATH = "models/url_detector_v5.pkl"
2 pipeline   = None
3
4 def load_model(path: str = MODEL_PATH):
5     global pipeline
6     pipeline = joblib.load(path)
```

```

7     return pipeline
8
9 def predict_urls(urls: list) -> list:
10     series = pd.Series(urls)
11     preds = pipeline.predict(series)
12     probas = pipeline.predict_proba(series)
13
14     results = []
15     for url, pred, prob in zip(urls, preds, probas):
16         results.append({
17             "url": url,
18             "result": "SAFE" if pred == "benign" else "
19                     DANGEROUS",
20             "confidence": round(max(prob) * 100, 2)
21         })
22     return results

```

Listing 6: Model storage path, loading function, and prediction engine.

1.5.1 Main Execution Workflow

The program execution begins within the `if __name__ == "__main__":` block. At startup, the system automatically creates the required directories (`models/` and `reports/`) if they do not already exist.

To improve efficiency, the system first checks whether a trained model already exists at `MODEL_PATH`. If the model is available, it is loaded directly. Otherwise, the training process is executed.

1.5.2 Dataset Selection and Loading

A graphical file selection interface is implemented using the Tkinter library. The file dialog allows users to select datasets without manually entering file paths. The interface is configured to display only the file selection window and keep it visible above other applications.

The system supports both CSV and Excel (`.xlsx`) datasets. For CSV files, the `latin1` encoding is used to improve compatibility with non-standard characters and multilingual URL datasets.

1.5.3 Data Cleaning and Preprocessing

Before model training, several preprocessing steps are applied to improve data quality:

- Standardizing column names to lowercase.
- Renaming the `type` column to `label` when necessary.
- Removing missing values using `dropna()`.
- Converting URLs to lowercase and removing unnecessary whitespace.
- Removing duplicate URLs to prevent data leakage and overfitting.
- Filtering invalid entries by enforcing a minimum URL length requirement.

These steps ensure that the dataset is consistent, clean, and suitable for machine learning.

1.5.4 Dataset Splitting

The cleaned dataset is divided into training and testing subsets using Scikit-learn's `train_test_split()` function. Key parameters include:

- `test_size=0.2`: 80% of the data is used for training, while 20% is reserved for evaluation.
- `random_state=42`: Ensures reproducible results across multiple runs.
- `stratify=y`: Preserves the original class distribution between benign and phishing URLs in both subsets.

Maintaining class balance is particularly important in malicious URL detection, as it prevents biased evaluation and ensures that both classes are adequately represented during training and testing.

1.6 Feature Extraction using FeatureUnion and TF-IDF

To convert URLs into machine-readable numerical representations, the proposed system employs two independent TF-IDF feature extraction pipelines. The first pipeline focuses on the domain component, while the second pipeline analyzes the URL path. Both pipelines are later combined using Scikit-learn's `FeatureUnion`.

```
1 domain_tfidf = Pipeline([
2     ("extract", DomainExtractor()),
3     ("tfidf", TfidfVectorizer(
4         analyzer="word",
5         token_pattern=r"(?u)\S+",
6         sublinear_tf=True,
7         min_df=2,
```

```
8         max_df=0.95 ,
9         max_features=10000 ,
10        ngram_range=(1, 2)
11    ))
12 ])
13
14 path_tfidf = Pipeline([
15     ("extract", PathExtractor()),
16     ("tfidf", TfidfVectorizer(
17         analyzer="word",
18         token_pattern=r"(?u)\S+",
19         sublinear_tf=True,
20         min_df=2,
21         max_df=0.95,
22         max_features=5000,
23         ngram_range=(1, 2)
24     ))
25 ])
26
27 features = FeatureUnion([
28     ("domain", domain_tfidf),
29     ("path", path_tfidf)
30 ])
```

Listing 7: Domain and path TF-IDF pipelines combined via FeatureUnion.

The feature extraction process is intentionally divided into two separate pipelines. The domain component contains the most important phishing indicators, such as suspicious keywords, abnormal top-level domains, and brand impersonation patterns. Therefore, it is allocated a larger feature space of 10,000 TF-IDF features.

In contrast, the path component mainly contains directory structures and query parameters. Since legitimate websites may frequently include words such as `login`, `secure`, or `update` within their paths, the path analysis focuses on structural information rather than sensitive keyword detection. This design helps reduce false-positive classifications.

Both pipelines utilize the TF-IDF algorithm to convert textual tokens into numerical vectors. Several parameter optimizations are applied:

- `sublinear_tf=True` – reduces the impact of repeated terms.
- `min_df=2` – removes extremely rare tokens.
- `max_df=0.95` – eliminates overly common terms.

- `ngram_range=(1,2)` – allows the model to learn both individual tokens and token pairs, enabling the detection of contextual patterns that may indicate malicious behavior.

Finally, the `FeatureUnion` component merges the outputs of the domain and path pipelines into a single feature matrix. As a result, each URL is represented by a comprehensive set of up to 15,000 numerical features, providing the machine learning classifiers with detailed information for accurate phishing URL detection.

1.7 Logistic Regression Classifier

1.7.1 Foundation: How the Algorithm Interprets URLs

As described in the preprocessing stage, after passing through the `FeatureUnion` framework, each URL is transformed into a feature vector containing up to 15,000 numerical values. Most of these values are zero, while a small subset contains TF-IDF scores corresponding to specific tokens or features extracted from the URL.

Let these TF-IDF values be represented as:

$$x_1, x_2, \dots, x_{15000}$$

For example:

- x_1 may represent the TF-IDF score of the token `__kw_login__`.
- x_2 may represent the TF-IDF score of the token `__tld_ru__`.
- x_3 may represent a benign token such as `article`.

1.7.2 Step 1: Computing the Linear Risk Score

The primary objective of Logistic Regression is to learn a weight (w) for each feature. These weights indicate how strongly a feature contributes to the phishing classification decision:

- A large positive weight suggests that the feature is strongly associated with malicious URLs.
- A negative weight suggests that the feature is commonly associated with legitimate URLs.
- A weight close to zero indicates that the feature has little influence on the prediction.

For each URL, the model computes a linear score z as follows:

$$z = w_1x_1 + w_2x_2 + \cdots + w_{15000}x_{15000} + b$$

where b is the bias term.

In practice, the model may learn that the token `__kw_login__` should receive a large positive weight because it frequently appears in phishing URLs, whereas a common word such as `article` may receive a negative or near-zero weight. Consequently, URLs containing suspicious tokens will produce higher risk scores.

1.7.3 Step 2: Converting the Score into a Probability

The linear score z can take any real value, ranging from highly negative to highly positive. However, the ensemble framework requires a probability value between 0 and 1. Therefore, Logistic Regression applies the Sigmoid function:

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

where $\hat{y}$ represents the probability that the URL is malicious.

The behavior of the Sigmoid function can be interpreted as follows:

- If $z = 0$, then $\hat{y} = 0.5$, indicating maximum uncertainty.
- If z is a large positive value, $\hat{y}$ approaches 1, indicating a high probability of phishing.
- If z is a large negative value, $\hat{y}$ approaches 0, indicating a benign URL.

This probability is subsequently used by the `predict_proba()` function and later contributes to the Soft Voting ensemble decision.

1.7.4 Step 3: Model Training Using Log-Loss and L-BFGS

At the beginning of training, the model assigns random values to the weights. It then evaluates the quality of its predictions using the Binary Cross-Entropy Loss (Log-Loss) function:

$$J(w, b) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$$

where y is the true class label, $\hat{y}$ is the predicted probability, and m is the total number of training samples.

The objective is to minimize this loss function by continuously updating the weights. In this project, optimization is performed using the **L-BFGS** algorithm, specified by

the parameter `solver="lbfgs"`.

```
1 log_reg = LogisticRegression(  
2     solver="lbfgs",  
3     max_iter=5000,  
4     C=1.0,  
5     random_state=42,  
6     class_weight="balanced"  
7 )
```

Listing 8: Logistic Regression classifier configuration.

The optimization process iteratively adjusts the model parameters until convergence is achieved or the maximum number of iterations (`max_iter=5000`) is reached.

1.7.5 Parameter Configuration

The selected parameters play a significant role in controlling the learning behavior of the model:

- **C = 1.0** – controls the strength of L2 regularization. Regularization prevents the model from assigning excessively large weights to individual features, thereby reducing the risk of overfitting and improving generalization performance.
- **class_weight = "balanced"** – automatically adjusts class weights according to the distribution of benign and phishing URLs. This is particularly important when the dataset is imbalanced, as it forces the model to pay greater attention to the minority phishing class during training.
- **solver = "lbfgs"** – specifies the optimization algorithm used to minimize the loss function.
- **max_iter = 5000** – allows sufficient optimization iterations for convergence when working with high-dimensional TF-IDF feature vectors.
- **random_state = 42** – ensures that the training process remains reproducible across multiple executions.

Overall, Logistic Regression serves as a strong linear classifier capable of learning global relationships among thousands of URL features and producing reliable probability estimates for the final ensemble prediction.

1.8 LightGBM Classifier

The second base classifier employed in the proposed ensemble framework is the Light Gradient Boosting Machine (LightGBM). Within the Voting Classifier architecture, LightGBM is assigned a higher voting weight than Logistic Regression, indicating that its prediction contributes more significantly to the final classification decision.

```

1 lgbm = LGBMClassifier(
2     random_state=42,
3     n_jobs=-1,
4     verbose=-1,
5     learning_rate=0.05,
6     n_estimators=1000,
7     num_leaves=63,
8     max_depth=8,
9     min_child_samples=20,
10    subsample=0.8,
11    subsample_freq=1,
12    colsample_bytree=0.8,
13    reg_alpha=0.1,
14    reg_lambda=1.0,
15    class_weight="balanced"
16 )

```

Listing 9: LightGBM classifier configuration.

1.8.1 Foundation: Gradient Boosting

Unlike Logistic Regression, which relies on a single mathematical model, LightGBM is an ensemble learning algorithm composed of hundreds or even thousands of decision trees. It is built upon the Gradient Boosting framework, where each newly generated tree focuses on correcting the mistakes made by previous trees.

The training process begins with an initial decision tree that attempts to classify URLs as benign or phishing. The model then measures the prediction errors and constructs a new tree specifically designed to reduce those errors. This iterative process continues until the predefined number of trees is reached.

Mathematically, the boosting process can be represented as:

$$F_m(x) = F_{m-1}(x) + \nu \cdot h_m(x)$$

where:

- x represents the URL feature vector generated from TF-IDF extraction.
- $F_{m-1}(x)$ denotes the prediction of the existing ensemble.
- $h_m(x)$ represents the newly generated decision tree that learns from the residual errors.
- ν is the learning rate, specified in this project as `learning_rate = 0.05`.

This mechanism allows the model to gradually improve its predictive capability by continuously focusing on difficult samples that were previously misclassified.

1.8.2 Key Innovations of LightGBM

LightGBM introduces several algorithmic improvements that make it particularly suitable for high-dimensional sparse TF-IDF datasets.

Leaf-wise Tree Growth Traditional tree-based algorithms commonly expand trees level by level. In contrast, LightGBM adopts a leaf-wise growth strategy, which always expands the leaf node associated with the largest reduction in loss. As a result, the model allocates more computational effort to complex phishing URLs that are difficult to classify, rather than spending resources on URLs that are already correctly identified.

Exclusive Feature Bundling (EFB) The TF-IDF representation used in this project generates up to 15,000 features, most of which are sparse and contain zero values. To improve memory efficiency, LightGBM applies Exclusive Feature Bundling (EFB), which automatically combines mutually exclusive features into shared feature groups without losing information. This significantly reduces memory consumption and training time while maintaining predictive performance.

Histogram-based Learning Instead of evaluating every possible numerical split point, LightGBM discretizes continuous feature values into histogram bins. Tree construction is then performed on these histogram representations rather than the original feature values. This approach greatly accelerates training and enables LightGBM to efficiently process large-scale URL datasets.

1.8.3 Parameter Configuration

Several hyperparameters were carefully selected to balance predictive performance and model generalization:

- **learning_rate = 0.05** – controls the contribution of each new tree. A relatively small learning rate allows the model to learn gradually and reduces the risk of

overfitting.

- **n_estimators = 1000** – specifies that up to 1,000 decision trees may be constructed. The combination of a small learning rate and a large number of trees enables more stable learning.
- **max_depth = 8** – limits the maximum depth of each tree, preventing overly complex decision structures.
- **num_leaves = 63** – controls the maximum number of leaf nodes. Although a tree with depth 8 could theoretically produce 256 leaves, restricting this to 63 encourages more selective branching and improves generalization.
- **min_child_samples = 20** – requires each leaf node to contain at least 20 training samples before further splitting. This helps eliminate noise and prevents the model from learning patterns that appear only in a very small number of URLs.
- **subsample = 0.8** and **colsample_bytree = 0.8** – introduce randomness by using only a subset of training samples and features during tree construction, helping reduce overfitting and improve model robustness.
- **reg_alpha = 0.1** and **reg_lambda = 1.0** – apply L1 and L2 regularization, respectively, to prevent excessively complex decision trees.
- **class_weight = "balanced"** – automatically adjusts class weights according to the distribution of phishing and benign URLs, ensuring that the minority phishing class receives sufficient attention during training.

1.8.4 Role in the Ensemble Framework

While Logistic Regression captures linear relationships among URL features, LightGBM specializes in learning complex non-linear interactions between tokens, domain characteristics, and URL structures.

For example, Logistic Regression may recognize that a keyword such as *login* is suspicious, whereas LightGBM can learn more sophisticated patterns involving combinations of suspicious keywords, domain structures, path depth, and parameter configurations.

Consequently, LightGBM serves as the primary non-linear classifier in the proposed ensemble architecture, complementing the strengths of Logistic Regression. The probability outputs generated by both models are subsequently combined through a Soft Voting mechanism to produce the final phishing detection decision.

1.9 Ensemble Classification using Soft Voting

The final stage of the proposed detection framework employs a **VotingClassifier**, which acts as an ensemble decision-making mechanism. Unlike Logistic Regression and LightGBM, the Voting Classifier does not directly analyze URL features. Instead, it aggregates the predictions generated by the individual classifiers and produces a final decision.

This approach belongs to a machine learning paradigm known as **Ensemble Learning**, which is based on the principle that combining multiple models often produces more reliable predictions than relying on a single classifier.

```

1 ensemble = VotingClassifier(
2     estimators=[("lr", log_reg), ("lgbm", lgbm)],
3     voting="soft",
4     weights=[1, 2]
5 )

```

Listing 10: Soft Voting ensemble combining Logistic Regression and LightGBM.

1.9.1 Soft Voting Mechanism

Two common voting strategies are typically used in ensemble learning:

- **Hard Voting:** The final prediction is determined by the majority vote of the participating classifiers.
- **Soft Voting:** The strategy adopted in this project. Instead of considering only the final class labels, each classifier provides a probability estimate, and the ensemble combines these probabilities to obtain the final prediction.

The Soft Voting process can be mathematically expressed as a weighted average:

$$P = \frac{w_1 \cdot p_{lr} + w_2 \cdot p_{lgbm}}{w_1 + w_2}$$

where:

- P is the final phishing probability generated by the ensemble model.
- p_{lr} is the phishing probability predicted by Logistic Regression.
- p_{lgbm} is the phishing probability predicted by LightGBM.
- w_1 and w_2 represent the weights assigned to each classifier.

1.9.2 Weighted Decision Strategy

In the proposed architecture, the ensemble uses the weight configuration `weights=[1, 2]`, assigning a weight of 1 to Logistic Regression and a weight of 2 to LightGBM. Consequently, the prediction generated by LightGBM contributes twice as much to the final decision.

For example, consider a suspicious URL evaluated by both classifiers:

- Logistic Regression predicts a phishing probability of 0.30.
- LightGBM predicts a phishing probability of 0.90.

The ensemble probability is calculated as:

$$P = \frac{1 \times 0.30 + 2 \times 0.90}{1 + 2} = \frac{2.10}{3} = 0.70$$

As a result, the final phishing probability becomes 70%. Since this value exceeds the classification threshold of 0.5, the URL is classified as malicious.

1.9.3 Rationale for Combining Logistic Regression and LightGBM

The combination of Logistic Regression and LightGBM was selected to leverage the complementary strengths of both algorithms.

Logistic Regression is effective at identifying direct linear relationships between individual URL features and phishing behavior. It performs particularly well when suspicious indicators are explicitly present within the URL structure.

In contrast, LightGBM excels at discovering complex non-linear interactions among multiple features. Through its decision-tree architecture, it can capture subtle relationships between domain characteristics, URL structure, path depth, and other extracted features that may not be detectable through linear models.

By integrating both classifiers into a Soft Voting ensemble, the system reduces the limitations associated with relying on a single algorithm. Logistic Regression provides stable and interpretable predictions, while LightGBM contributes powerful non-linear pattern recognition capabilities.

1.9.4 Benefits of the Ensemble Approach

The proposed ensemble architecture offers several advantages:

- Improved robustness against individual model weaknesses.
- Better generalization to previously unseen URLs.

- Reduced risk of misclassification caused by model-specific biases.
- Lower false-positive rates through probability averaging.
- Enhanced overall detection performance by combining linear and non-linear learning mechanisms.

Overall, the Voting Classifier serves as the final decision layer of the system, combining the strengths of Logistic Regression and LightGBM to produce a more accurate and reliable phishing URL detection model.

1.10 Pipeline Integration and Model Evaluation

1.10.1 Pipeline Construction

To streamline the entire machine learning workflow, the project integrates feature extraction and classification into a single Scikit-learn Pipeline.

```
1 pipeline = Pipeline([
2     ("features", features),
3     ("ensemble", ensemble)
4 ])
```

Listing 11: End-to-end pipeline integrating feature extraction and ensemble classification.

The primary objective of this design is to create an end-to-end processing framework that automatically executes every stage of the detection process. Instead of manually performing feature extraction, TF-IDF transformation, and classification separately, the pipeline encapsulates all operations into a single workflow.

As a result, a raw URL can be directly passed to the system through `pipeline.predict()`, and the framework automatically performs tokenization, feature extraction, vectorization, and classification without requiring additional user intervention. This approach improves maintainability, reduces implementation complexity, and minimizes the risk of human error during prediction.

1.10.2 Cross-Validation

Before training the final model on the complete training dataset, a Cross-Validation procedure is performed to evaluate model stability and generalization capability.

```
1 cv = StratifiedKFold(
2     n_splits=3,
3     shuffle=True,
```



```
4     random_state=42
5 )
```

Listing 12: Stratified K-Fold Cross-Validation configuration.

The system employs **Stratified K-Fold Cross-Validation** with three folds. The validation dataset is divided into three equally sized subsets while preserving the original class distribution between benign and phishing URLs.

During each iteration, two subsets are used for training and one subset is reserved for validation. This process is repeated three times, allowing every sample to participate in both training and validation.

The purpose of this procedure is to reduce evaluation bias caused by a single train-test split. By assessing the model across multiple validation folds, the system obtains a more reliable estimate of its true performance and can identify potential issues before the computationally expensive full-scale training stage begins.

1.10.3 Model Training and Persistence

After successful validation, the complete pipeline is trained using the training dataset:

```
1 pipeline.fit(X_train, y_train)
2 joblib.dump(pipeline, MODEL_PATH)
```

Listing 13: Model training and serialization.

During this phase, the TF-IDF vectorizers learn feature representations from the URLs, Logistic Regression estimates its parameters, and LightGBM constructs and optimizes its ensemble of decision trees.

Once training is completed, the entire pipeline is serialized and stored on disk. Saving the trained model enables future predictions without repeating the training process. Subsequent executions can simply load the serialized model, significantly reducing startup time and computational cost.

1.10.4 Performance Evaluation

Following training, the system performs a comprehensive evaluation using the testing dataset. One of the most important evaluation tools employed is the **Confusion Matrix**, which provides a detailed breakdown of classification outcomes. The confusion matrix identifies:

- **True Positives (TP)**: phishing URLs correctly classified as phishing.
- **True Negatives (TN)**: benign URLs correctly classified as benign.

- **False Positives (FP):** benign URLs incorrectly classified as phishing.
- **False Negatives (FN):** phishing URLs incorrectly classified as benign.

Unlike overall accuracy, the confusion matrix provides deeper insights into the types of mistakes made by the model and helps guide future optimization efforts.

1.10.5 False Positive Analysis

Particular attention is given to the analysis of False Positives. The system automatically identifies URLs whose actual label is *benign* but are predicted as *phishing*. These misclassified samples are exported into a dedicated report file `misclassified_v5.csv`.

This analysis is especially important in cybersecurity applications. Incorrectly blocking a legitimate banking website, government portal, or business service may create significant inconvenience for users and reduce trust in the detection system.

By examining the False Positive report, developers can identify problematic patterns and refine the preprocessing rules, suspicious keyword lists, domain heuristics, and feature engineering strategies used by the model.

1.11 Deployment and Model Loading

After training, the model can be deployed and accessed through external applications. A practical issue encountered during deployment involves loading serialized Scikit-learn pipelines that contain custom transformer classes.

```
1 import __main__
2
3 __main__.DomainExtractor = Tuna1.DomainExtractor
4 __main__.PathExtractor = Tuna1.PathExtractor
```

Listing 14: Re-registering custom transformer classes for deployment compatibility.

When the model is serialized using Pickle or Joblib, references to custom classes such as `DomainExtractor` and `PathExtractor` are stored within the model file. During deployment, if these classes cannot be located in the current execution environment, the loading process fails.

To address this issue, the deployment script explicitly re-registers the custom classes before loading the model. This ensures that the serialized pipeline can correctly reconstruct all required components and operate without runtime errors.

Summary

The proposed system implements a complete machine learning pipeline for phishing URL detection, encompassing data preprocessing, feature extraction, ensemble learning, cross-validation, model persistence, performance evaluation, and deployment support. By combining cybersecurity-oriented feature engineering with advanced machine learning techniques such as TF-IDF vectorization, Logistic Regression, LightGBM, and Soft Voting, the framework provides a robust and scalable solution for malicious URL detection.

1.12 Model Deployment and Testing

The `Test.py` file serves as the deployment and inference environment of the proposed system. While `Tuna1.py` is responsible for data preprocessing, model training, and model persistence, `Test.py` demonstrates how an external application can load the trained model and perform real-time URL classification.

1.12.1 Environment Setup

The script first imports the required modules and configures the Python search path to ensure that the main project module can be located and loaded correctly.

```
1 import sys
2 import __main__
3
4 sys.path.append(r"D:\New folder")
5 import Tuna1
```

Listing 15: Environment setup and module import in `Test.py`.

The `sys.path.append()` statement explicitly adds the project directory to Python's module search path. This allows the deployment script to access the functions and classes defined in `Tuna1.py`, even when the files are located in different directories.

1.12.2 Handling Pickle Deserialization Issues

Before loading the trained model, the script applies a compatibility fix for custom transformer classes used during preprocessing.

```
1 __main__.DomainExtractor = Tuna1.DomainExtractor
2 __main__.PathExtractor = Tuna1.PathExtractor
```

Listing 16: Registering custom classes to resolve Pickle deserialization errors.

The trained pipeline was previously serialized using `joblib.dump()`, which internally relies on Python's Pickle mechanism. During serialization, Pickle stores references to custom classes such as `DomainExtractor` and `PathExtractor`.

When `Test.py` is executed, the active module becomes `__main__`, which differs from the environment in which the model was originally saved. Consequently, the deserialization process may fail because Pickle cannot locate the required custom classes. To resolve this issue, the script explicitly registers both classes within the current `__main__` namespace before loading the model, ensuring that the serialized pipeline can be reconstructed successfully without runtime errors.

1.12.3 Model Loading and Validation

The trained model is loaded into memory and subsequently validated through the following commands:

```
1 Tuna1.load_model()
2
3 if not hasattr(Tuna1, "predict_urls"):
4     raise AttributeError(...)
```

Listing 17: Model loading and function availability check.

This safety check verifies that the `predict_urls()` function is available and accessible. If the function cannot be found, the script immediately raises an informative error message, helping developers identify implementation issues before the prediction stage begins.

1.12.4 URL Classification Process

A collection of test URLs is prepared and passed to the prediction function:

```
1 urls = [
2     "https://facebook.com",
3     "http://paypal-security-update-login.net",
4     ...
5 ]
6
7 for r in Tuna1.predict_urls(urls):
8     flag = "OK" if r["result"] == "SAFE" else "NO"
9     print(
10         f"{flag} {r['result']:<10} "
11         f"({r['confidence']:5.1f}%) "
12         f"{r['url']}"
```

13

)

Listing 18: URL classification and formatted output.

For every URL, the model returns:

- The predicted class label (**SAFE** or **DANGEROUS**).
- The confidence score generated from the ensemble model.
- The original URL being evaluated.

The output is formatted using visual indicators to improve readability, enabling users to quickly identify potentially malicious URLs.

Summary

The `Test.py` script demonstrates the practical deployment stage of the phishing URL detection system. It provides a lightweight inference interface capable of loading the trained model, handling serialization dependencies, validating required functions, and performing real-time URL classification. This deployment workflow illustrates how the trained machine learning model can be integrated into external cybersecurity applications such as browser extensions, endpoint protection software, or network monitoring systems.

1.13 Experimental Results and Performance Analysis

1.13.1 Overall Model Performance

The proposed ensemble model achieved an overall Accuracy of **84.97%**, correctly classifying approximately 85% of the 128,640 URLs in the testing dataset. While this result indicates good predictive performance, accuracy alone does not fully reflect model effectiveness due to the imbalance among URL categories.

A more representative metric is the Weighted One-vs-Rest (OvR) AUC-ROC, which reached **0.9713**. This value demonstrates that the model possesses excellent discriminative capability and can effectively distinguish between benign and malicious URLs. Therefore, the feature extraction process and ensemble learning architecture successfully captured the underlying characteristics of different URL categories.

1.13.2 Class-Level Performance Analysis

Table 1 presents the classification results for each URL category.

Table 1: Classification Report of the Proposed Ensemble Model

Class	Precision	Recall	F1-score	Support
Benign	0.97	0.80	0.88	85,612
Defacement	0.97	0.98	0.97	19,053
Malware	0.95	0.95	0.95	4,729
Phishing	0.51	0.90	0.65	19,246
Accuracy			0.85	128,640
Macro avg	0.85	0.91	0.86	128,640
Weighted avg	0.90	0.85	0.86	128,640
Accuracy: 84.97% AUC-ROC (Weighted OvR): 0.9713				

Benign URLs The benign class achieved a Precision of 0.97, indicating that 97% of URLs predicted as benign were actually legitimate websites. This demonstrates a high level of confidence when the model classifies a URL as safe. However, the Recall value of 0.80 suggests that only 80% of all benign URLs were correctly identified. Consequently, approximately 20% of legitimate websites were incorrectly classified as malicious, contributing significantly to the overall false-positive rate.

Defacement URLs The defacement category achieved outstanding performance with a Precision of 0.97, Recall of 0.98, and F1-score of 0.97. These results indicate that the model successfully learned the distinguishing characteristics of defacement URLs and rarely misclassified them.

Malware URLs Similarly, the malware category achieved a Precision of 0.95, Recall of 0.95, and F1-score of 0.95. The balanced performance across all evaluation metrics demonstrates the model’s strong ability to identify malware-related URLs while maintaining a low error rate.

Phishing URLs The phishing category represents the primary challenge of the proposed system. The model achieved a Recall of 0.90, meaning that 90% of phishing URLs were successfully detected. From a cybersecurity perspective, this is a highly desirable result because the system is able to identify the vast majority of malicious phishing attempts.

However, the Precision value is only 0.51, indicating that nearly half of the URLs classified as phishing were actually benign. In practical terms, among 100 URLs flagged as phishing, only 51 are truly phishing websites, while the remaining 49 are false alarms. This behavior suggests that the model is highly sensitive to phishing-related features and tends to over-classify suspicious URLs.

1.13.3 Discussion

The experimental results indicate that the proposed ensemble architecture successfully captures the distinguishing characteristics of malware, defacement, and phishing URLs. The high AUC-ROC score of 0.9713 further confirms the effectiveness of the TF-IDF feature extraction process combined with Logistic Regression and LightGBM.

Nevertheless, the phishing category exhibits a noticeable imbalance between Precision and Recall. The model prioritizes detecting phishing attacks (high Recall) at the cost of generating a relatively large number of false positives (low Precision). While this trade-off is acceptable in many cybersecurity scenarios, excessive false alarms may negatively affect user experience and system usability.

1.13.4 Future Improvements

Several optimization strategies can be considered to improve phishing detection performance:

- **Threshold Tuning:** Instead of using the default decision threshold, a higher probability threshold may be applied to phishing predictions in order to reduce false-positive rates while maintaining an acceptable recall level.
- **Domain Whitelisting:** Incorporating a whitelist of trusted domains can prevent legitimate websites from being incorrectly classified as phishing.
- **Feature Refinement:** Additional URL features, domain reputation indicators, and lexical characteristics may be incorporated to improve phishing discrimination.
- **Probability Calibration:** Calibration techniques such as Platt Scaling or Isotonic Regression can be employed to produce more reliable probability estimates and improve classification confidence.

Overall, the proposed model demonstrates strong classification capability and achieves particularly high performance in malware and defacement detection, while future improvements should focus primarily on increasing the precision of phishing detection.